Dynamic Attentional Context Scoping: Agent-Triggered Focus Sessions for Isolated Per-Agent Steering in Multi-Agent LLM Orchestration

Nickson Patel
Independent Researcher
nicksonpatel@gmail.com

Abstract

Multi-agent LLM orchestration systems suffer from *context pollution*: when N concurrent agents compete for the orchestrator's context window, each agent's task state, partial outputs, and pending questions contaminate the steering interactions of every other agent, degrading decision quality. We introduce **Dynamic Attentional Context Scoping (DACS)**, a mechanism in which the orchestrator operates in two asymmetric modes. In REGISTRY mode it holds only lightweight per-agent status summaries (≤ 200 tokens each), remaining responsive to all agents and the user. When an agent emits a STEERINGREQUEST, the orchestrator enters FOCUS(a_i) mode, injecting the full context of agent a_i while compressing all other agents to their registry entries. Context isolation is agent-triggered, asymmetric, and deterministic: the context window contains exactly $F(a_i) + R_{-i}$ during steering, eliminating cross-agent contamination without requiring context compression or retrieval. We evaluate DACS across five experimental phases totalling 260 trials: Phase 1 tests $N \in \{3, 5, 10\}$ agents (60 trials); Phase 2 tests agent heterogeneity and adversarial dependency structures (60 trials); Phase 3 tests decision density scaling up to $D = 15$ steering requests per agent (40 trials); Phase 4 replaces all scripted stubs with autonomous LLM agents generating free-form questions (40 trials, Claude Haiku 4.5); Phase 5 evaluates Orchestrator-Initiated Focus (OIF) at $N \in \{3, 5\}$ (60 trials). We formalise **OIF**, extending the state machine with three top-down transitions (T5–T7) for proactive focus triggering via user-query routing, agent lifecycle signals, and iteration relay, completing DACS as a fully bidirectional mechanism. Across all 8 synthetic scenarios, DACS achieves 90.0–98.4% steering accuracy versus 21.0–60.0% for a flat-context baseline ($p < 0.0001$ throughout), with wrong-agent contamination falling from 28–57% to 0–14% and context efficiency ratios of up to $3.53\times$. The accuracy advantage grows with both agent count (N) and decision density (D), and keyword matching is validated by LLM-as-judge across all three synthetic phases (mean $\kappa = 0.909$). In Phase 4, all scripted stubs are replaced by autonomous LLM agents (Claude Haiku 4.5) generating free-form steering questions: DACS outperforms the flat-context baseline by $+17.2$ pp at N=3 (p=0.0023) and $+20.4$ pp at N=5 (p=0.0008), with the advantage growing with N confirmed by two independent judges. In Phase 5 (OIF evaluation), OIF achieves 100% M1_{T5} user-query routing accuracy versus 28–40% for DACS without OIF and 28–33% for the flat baseline at $N \in \{3, 5\}$ ($p < 0.0001$, Mann–Whitney), using 2.1–$3.1\times$ fewer context tokens than flat context. All code and data are released at github.com/nicksonpatel/dacs-agent-focus-mode.

Preprint. arXiv submission.

1 Introduction

The deployment of LLMs as orchestrators managing multiple parallel agents has become a practical reality: systems such as Claude Code Agent Teams [Anthropic, 2025], OpenCode [Anomaly, 2025], and production multi-agent platforms [Zhang et al., 2025] demonstrate that complex long-horizon tasks can be decomposed across concurrent specialized agents. The common architectural choice is a single LLM instance (the *orchestrator*) that coordinates all agents, handles user interaction, and steers individual agents when they reach decision points.

The context pollution problem. This architecture introduces a fundamental scaling problem. When N agents run concurrently, each maintains its own task context: partial outputs, domain-specific state, pending decisions. In a flat-context orchestrator, all of these threads compete for space in a single context window. When agent a_i requests steering—"should I use BFS or DFS for this graph traversal?"—the orchestrator's context simultaneously holds a_j's transformer attention survey, a_k's CSV encoding problem, and so on. We call this *context pollution*: the systematic contamination of one agent's steering interaction by irrelevant context from other agents.

The consequences are measurable and severe. Prior work has documented context pollution in single-agent settings under tool overload [Sampath and Baskaran, 2025, Fei et al., 2025], under retrieval bloat [Kariyappa and Suh, 2025], and under long conversation histories [Cruz, 2024, Zhang et al., 2026]. Our experiments show that in the multi-agent setting, a flat-context orchestrator's steering accuracy collapses from 60% at $N = 3$ to 21% at $N = 10$ (Phase 1), and further degrades as agents become more diverse (Phase 2) or accumulate longer decision histories (Phase 3).

The DACS mechanism. We propose **Dynamic Attentional Context Scoping (DACS)**, which solves context pollution through *agent-triggered asymmetric context isolation*. DACS introduces two orchestrator modes:

- **REGISTRY mode.** The orchestrator holds only the registry $R = \{r_1, \ldots, r_N\}$, one compact status summary per agent (≤ 200 tokens each). It monitors all agents, responds to the user, and queues incoming steering requests.
- **FOCUS(a_i) mode.** When agent a_i emits a STEERINGREQUEST, the orchestrator transitions to hold $F(a_i)$ (the full context of a_i: task description, prior steering exchanges, current partial output) plus a compressed version of R_{-i} (registry summaries for all agents except a_i). The context window contains exactly what is needed to steer a_i, and nothing from any other agent's task thread.

The key properties of DACS that distinguish it from prior context management approaches are:

1. **Agent-triggered.** Context isolation fires when an agent needs it, not on every turn or at task-decomposition time.
2. **Asymmetric.** The steered agent gets full fidelity; all others get compressed summaries.
3. **Exact, not approximate.** No scoring function, no fidelity tiering [Cruz, 2024, Jiang et al., 2025], no probabilistic compression: the context is deterministically constructed as $F(a_i) + R_{-i}$.
4. **Sub-linear in N and D.** Focus context size is independent of the number of other agents; only the compressed registry grows with N (by ≈ 25 tokens per additional agent in our experiments). Accuracy is also stable as decision density D increases.
5. **Bidirectional.** While agents trigger focus sessions bottom-up via STEERINGREQUEST, OIF adds top-down triggering: the orchestrator proactively enters FOCUS(a_i) for user-query routing, agent lifecycle signals, and iteration relay, without waiting for an explicit agent request.

Contributions.

1. We formally define the DACS mechanism: the REGISTRY/ FOCUS state machine, the STEERINGREQUEST protocol, and the context builder invariants (§3).

2. We implement a minimal, fully observable orchestration harness ($\sim$300 lines) where every token entering each LLM call is logged, enabling strict experimental control (§4).
3. We show large, statistically significant accuracy gains across Phase 1 (N scaling), Phase 2 (agent diversity and adversarial dependencies), and Phase 3 (decision density scaling), together answering four distinct research questions (§5).
4. We independently validate the keyword matching metric with LLM-as-judge evaluations across Phases 2 and 3 (Phase 2: $\kappa = 0.956$; Phase 3: mean $\kappa = 0.909$), confirming that reported advantage sizes are robust (§5).
5. We validate beyond the scripted benchmark with Phase 4: two real-agent experiments (40 trials, Claude Haiku 4.5, free-form questions) showing DACS outperforms the baseline by $+17.2$ pp and $+20.4$ pp at $N{=}3$ and $N{=}5$ respectively, with the advantage growing with N (§6).
6. We identify sub-linear context scaling as the mechanism's key theoretical property, explain its source, and discuss conditions under which it holds (§7).
7. We extend DACS with Orchestrator-Initiated Focus (OIF): three new state machine transitions (T5–T7) for top-down focus triggering via user-query routing, agent lifecycle signals (`[[DONE:]]`, `[[BLOCKED:]]`, `[[PROGRESS:]]`), and iteration relay, completing the mechanism as a fully bidirectional context-scoping system (§3.6).
8. We evaluate OIF empirically in Phase 5 ($N{\in}\{3, 5\}$, 60 trials): OIF achieves 100% $\mathrm{M1}_{\mathrm{T5}}$ query-routing accuracy versus 28–40% for DACS without OIF and 28–33% for a flat baseline ($p < 0.0001$, Mann–Whitney; §8).

2 Related Work

Context management for LLM agents has been studied at three levels of granularity: single-agent memory across turns, single-agent tool/retrieval bloat, and multi-agent orchestration. DACS addresses a problem at the third level that existing work at the first two levels does not reach.

Single-turn context compression. AFM [Cruz, 2024] assigns per-message fidelity tiers (Full/Compressed/Placeholder) based on a composite recency-relevance score and packs messages greedily under a token budget. It achieves 83.3% constraint-recall on its benchmark, validating that structured fidelity management outperforms naive truncation. ACE [Zhang et al., 2026] treats the agent's context as an evolving playbook of itemized bullets with utility counters, using incremental delta updates to avoid context collapse (they observe a 18,282-token context collapsing to 122 tokens under iterative rewriting, dropping accuracy below the no-adaptation baseline). Both AFM and ACE operate within a single agent's context history. Neither addresses N concurrent agents competing for an orchestrator context window, and neither models wrong-agent contamination as a failure mode.

Retrieval and cache bloat. SideQuest [Kariyappa and Suh, 2025] frames KV cache compression as an auxiliary task solved by the LRM itself, achieving 65% peak token reduction on single-agent deep research tasks. Lemon Agent [Jiang et al., 2025] applies three-tier progressive compression (full $\rightarrow$ compressed $\rightarrow$ summary) to a shared orchestrator context over task time in an orchestrator-worker system. Both treat context as a resource to compress; neither introduces asymmetric isolation triggered by an agent's request.

Context pollution in tool-heavy agents. Adaptive Orchestration [Sampath and Baskaran, 2025] identifies context pollution and attention decay as failure modes in monolithic agents loaded with too many tools, and proposes spawning specialist sub-agents (DMoE) to offload capability. CodeDelegator [Fei et al., 2025] separates planner and executor roles via Ephemeral-Persistent State Separation (EPSS): a persistent Delegator never receives execution traces; each Coder sub-task starts with a clean context. Both solve context pollution *architecturally* (at task decomposition time) for sequential delegation to one sub-agent at a time. DACS solves it *dynamically at runtime*, during the steering interaction itself, for N agents running simultaneously. The distinction matters: CodeDelegator's Delegator has no mechanism for handling the case where multiple concurrent Coders simultaneously need steering.

Table 1: Comparison of context management mechanisms.

System	N concurrent	Asym. mode switch	Agent-triggered	OIF	Contam. metric
AFM	×	×	×	×	×
ACE	×	×	×	×	×
AOI	✓	×	×	×	×
AgentOrchestra	✓	×	×	×	×
AdaptOrch	✓	×	×	×	×
CodeDelegator	×	×	×	×	×
Adaptive Orch.	×	×	×	×	×
Lemon Agent	✓	×	×	×	×
DACS (ours)	✓	✓	✓	✓	✓

Multi-agent orchestration context management. AOI [Bai et al., 2024] introduces a three-layer memory architecture for multi-agent IT operations, with a central Context Compressor transforming raw agent outputs into a compressed cache. Its Observer agent always holds a compressed aggregate of *all* agent activity simultaneously; there is no per-agent isolation. Agents execute and report results; they cannot signal the orchestrator for steering attention, and wrong-agent contamination is not modelled (AOI's failure mode is information loss from operational data volume). AgentOrchestra [Zhang et al., 2025] achieves SOTA on GAIA (89.04%) via hierarchical delegation: a planning agent routes sub-tasks to domain-specific sub-agents, bounding the planner's context footprint by converting global coordination into localized routing decisions. The paper explicitly notes that flat coordination "tends to accumulate irrelevant context"—independent validation of the problem DACS solves. However, hierarchical routing is a structural (pre-execution) partial solution: it reduces *how much* context enters the orchestrator, but agents still request the orchestrator's attention concurrently at runtime, and the orchestrator has one flat context window for those interactions. DACS and AgentOrchestra are complementary: AgentOrchestra reduces the initial context volume; DACS controls what the orchestrator holds at the moment of each steering decision. AdaptOrch [Yu, 2025] formalizes topology selection as the dominant performance variable as frontier LLMs converge, showing $+6.9$–9.8pp gains on SWE-bench/GPQA/HotpotQA from topology-aware routing. AdaptOrch makes one topology decision per task; DACS makes repeated context-switch decisions throughout execution.

Summary. No prior work implements agent-triggered asymmetric REGISTRY/ FOCUS mode switching for per-agent context isolation in concurrent multi-agent orchestration, nor the complementary Orchestrator-Initiated Focus (OIF) transitions for proactive top-down focus triggering. Table 1 summarises the key distinctions.

3 Dynamic Attentional Context Scoping (DACS)

3.1 Entities and Notation

Let O denote the orchestrator LLM, $A = \{a_1, \ldots, a_N\}$ the set of N concurrently running agents, and T the context window token budget. Each agent a_i has an associated *focus context* $F(a_i)$, the full set of information the orchestrator needs to steer a_i: task description, previous steering exchanges between O and a_i, and a_i's current partial output summary.

The *registry* $R = \{r_1, \ldots, r_N\}$ is a set of compact status snapshots, one per agent. Each entry r_i contains:

`agent_id`	identifier
`task`	task description (≤ 50 tokens)
`status`	RUNNING | BLOCKED | WAITING | COMPLETE
`last_output_summary`	≤ 100 tokens
`urgency`	LOW | MEDIUM | HIGH

Target budget per entry: ≤ 200 tokens. Full registry size for $N = 10$: $\leq 2{,}000$ tokens, leaving ample space for $F(a_i)$.

3.2 Orchestrator State Machine

The orchestrator operates as an explicit finite-state machine with three states:

REGISTRY. O holds R only. It monitors all agents, responds to user messages, and processes incoming STEERINGREQUESTs from the queue.

FOCUS(a_i). O holds $F(a_i)$ and a compressed view of R_{-i} (all registry entries except a_i's, since a_i's full context is already in $F(a_i)$). O steers a_i and cannot accept new steering requests until the focus session ends.

USERINTERACT. O responds to a user message using R only (same as REGISTRY mode, but explicitly user-facing). Queued steering requests are not processed until this state exits.

Transitions:

$$\text{REGISTRY} \rightarrow \text{FOCUS}(a_i): \quad a_i \text{ emits STEERINGREQUEST(urgency} = \text{any)} \tag{1}$$

$$\text{FOCUS}(a_i) \rightarrow \text{REGISTRY}: \quad \text{STEERINGCOMPLETE or STEERINGABANDONED} \tag{2}$$

$$\text{FOCUS}(a_i) \rightarrow \text{FOCUS}(a_j): \quad a_j \text{ emits STEERINGREQUEST(urgency} = \text{HIGH}),\ a_j \neq a_i \tag{3}$$

$$\text{REGISTRY} \rightarrow \text{USERINTERACT}: \quad \text{user message received} \tag{4}$$

Transition (3) (the *interrupt* protocol) allows a high-urgency agent to preempt an active focus session. On interrupt, the current partial steering state for a_i is saved and a new focus context for a_j is built. After a_j is steered, the orchestrator returns to REGISTRY and resumes a_i's interrupted session if it is still pending.

3.3 The SteeringRequest Protocol

When agent a_i reaches a decision point it cannot resolve autonomously, it emits:

```
SteeringRequest {
  agent_id:   str
  context:    str   -- relevant context excerpt
  question:   str   -- specific decision needed
  blocking:   bool  -- is a_i halted?
  urgency:    LOW | MEDIUM | HIGH
}
```

HIGH urgency can interrupt an active focus session (transition (3)). MEDIUM requests queue; the agent continues on a default path. LOW requests are batched.

3.4 Context Builder

The context builder constructs the orchestrator's prompt before each LLM call. It enforces a hard token budget T deterministically (token count is checked and enforced before every call; the LLM provider is never relied upon for truncation).

`build_focus_context`(a_i)**.** Returns $F(a_i) \,\|\, \texttt{compress}(R_{-i})$. If $|F(a_i)| + |R_{-i}| > T$, registry entries for lower-urgency agents are progressively truncated until the budget is met. a_i's focus context $F(a_i)$ is never truncated.

`build_registry_context`()**.** Returns R in full. Used during REGISTRY and USERINTERACT modes.

Invariants enforced by the context builder:

1. O is never simultaneously in FOCUS mode for more than one agent.
2. User messages are never dropped; they are queued during FOCUS and processed on the next REGISTRY entry.

3. R is always current within the last agent heartbeat (agents push a compact status update after each step).
4. $|C| \leq T$ holds before every LLM call without exception.

3.5 Complexity Analysis

Let $|F|$ denote the average focus context size (task-dependent, independent of N) and $|r|$ the average registry entry size (≤ 200 tokens).

REGISTRY mode context size. $|C_{\text{reg}}| = N \cdot |r|$, growing linearly in N. However, this mode is lightweight: the orchestrator makes no costly steering decisions here, only monitoring and routing.

FOCUS(a_i) mode context size. $|C_{\text{focus}}| = |F| + (N-1) \cdot |r|$. The focus context $|F|$ is independent of N. Only the compressed registry of $N-1$ agents adds $O(N)$ tokens. For large $|F|$ and small $|r|$ (our target design point), this is approximately $|F| + O(N \cdot |r|)$ where $|r| \ll |F|$.

In Phase 1 experiments, $|F| \approx 450$–550 tokens per agent and $|r| \approx 25$ tokens, giving $|C_{\text{focus}}| \approx 500 + 25N$, empirically confirmed: 561 tokens at $N = 3$, 633 at $N = 5$, 816 at $N = 10$ (+25.5 tokens/agent, $R^2 > 0.99$).

In Phase 3 (high decision density, $D = 15$), $|F|$ grows with the accumulated steering history, yielding $|C_{\text{focus}}| \approx 2{,}755$ tokens, higher than Phase 1 in absolute terms, but isolated to exactly a_i's history. The flat baseline accumulates *all* agents' histories simultaneously, reaching 6,573 tokens (2.39× ratio).

Efficiency ratio. $\rho(N) = |C_{\text{flat}}|/|C_{\text{focus}}| \approx N|F|/(|F| + N|r|)$. As $N \to \infty$, $\rho \to |F|/|r| \approx 20$–$22\times$ at our design point. Empirically, ρ grows from $2.12\times$ ($N = 3$) to $3.53\times$ ($N = 10$) in Phase 1, and reaches $3.24\times$ at $N = 5, D = 8$ in Phase 3.

3.6 Orchestrator-Initiated Focus (OIF)

The four transitions (1)–(4) model *agent-initiated* focus: the orchestrator enters FOCUS(a_i) because a_i emitted a STEERINGREQUEST. Production deployments expose three further trigger classes where the orchestrator should proactively enter a focus session *without* an explicit agent request. We formalise these as **Orchestrator-Initiated Focus (OIF)**, adding three transitions:

$$\text{REGISTRY} \rightarrow \text{FOCUS}(a_i): \quad \text{user message semantically matches } r_i \tag{5}$$

$$\text{REGISTRY} \rightarrow \text{FOCUS}(a_i): \quad a_i \text{ emits } \texttt{[[DONE:]]} \text{ or } \texttt{[[BLOCKED:]]} \tag{6}$$

$$\text{FOCUS}(a_i) \rightarrow \text{FOCUS}(a_i): \quad \text{orchestrator relays iteration directive to } a_i \tag{7}$$

T5: User-query routing. When the user asks about a specific agent's work (e.g., "how far is the research agent?"), the orchestrator, in REGISTRY mode, matches the query against registry entries using prefix-stem overlap on content words (agent name, task keywords, or domain terms with length > 3). If at least two task-description stems match message words, the orchestrator enters FOCUS(a_i) before responding, grounding its answer in a_i's full context rather than only its compact registry summary.

T6: Lifecycle signals. We extend the signal vocabulary with three lifecycle markers emitted by agents as free-text suffixes:

`[[DONE: summary]]`	milestone reached; orchestrator review warranted (triggers T6)
`[[PROGRESS: update]]`	routine step completion; registry update only, no OIF
`[[BLOCKED: reason]]`	agent cannot proceed; handled identically to STEERINGREQUEST (T6)

A `[[DONE:]]` or `[[BLOCKED:]]` emission triggers transition (6): the orchestrator enters FOCUS(a_i) to review a_i's completed work or unblock condition before issuing a next-task assignment or resolution. A `[[PROGRESS:]]` emission updates r_i and returns immediately to REGISTRY mode, keeping the footprint low for routine heartbeats.

T7: Iteration relay. While in FOCUS(a_i), the orchestrator may determine that a follow-up directive (from the user or from a pipeline dependency) is addressed to a_i itself. Transition (7) is a *self-loop*: the orchestrator relays the directive to a_i and continues in FOCUS(a_i) rather than returning to REGISTRY before the next exchange. This amortises the context-switching overhead for multi-turn iteration within a single agent's work thread.

Registry extension. To distinguish OIF sessions from agent-triggered ones in logs and judge evaluations, we add a `trigger_type` field to each registry entry:

`AGENT_STEERING`	a_i emitted STEERINGREQUEST (transitions T1, T3)
`USER_QUERY`	user query matched r_i (transition T5)
`COMPLETION`	a_i emitted `[[DONE:]]` or `[[BLOCKED:]]` (transition T6)
`ITERATION`	orchestrator relayed directive to a_i (transition T7)

Invariant preservation. OIF preserves all context builder invariants from §3.4: the focus context is still deterministically $F(a_i) + R_{-i}$, the token budget is enforced before every LLM call, and at most one agent is in focus at any time. The only change is the direction of the trigger: top-down (orchestrator-decided) rather than bottom-up (agent-requested).

4 Experimental Setup

4.1 Harness Design

We implement a minimal Python orchestration harness ($\sim$300 lines across `src/`) with full context observability: *every* token entering each LLM call is logged to a `.jsonl` trial file. This is the central experimental variable. We use no external orchestration framework (LangGraph, CrewAI, etc.); their context assembly internals would add uncontrolled noise.

The harness has four components: `registry.py` (per-agent state management), `protocols.py` (STEERINGREQUEST/RESPONSE dataclasses), `context_builder.py` (token-counted context construction), `orchestrator.py` (state machine + LLM call dispatch).

LLM backend. Phases 1–3 use MiniMax-M2.7 via an Anthropic-compatible API endpoint (context window 204,800 tokens). Phase 4 uses Claude Haiku 4.5 via OpenRouter (agents and orchestrator both use the same model, maintaining a single-model stack with no confound from model differences within each phase). Within each phase, DACS and the baseline use the identical model and endpoint.

4.2 Conditions

DACS. Orchestrator operates with full REGISTRY/ FOCUS mode switching as described in §3.

Baseline (flat context). All agents' full contexts are injected simultaneously into every steering call. The baseline uses the *identical* code path as DACS, with the single difference that `build_focus_context` is replaced by `build_flat_context` (concatenates all $F(a_i)$). No other differences.

4.3 Task Suite

We design 8 scenarios across three experimental phases, each with known-correct answers defined per decision point. Correctness is evaluated by keyword matching: a steering response scores correct if it contains ≥ 1 expected ground-truth keyword for agent a_i's current decision point. Each agent's correct-answer keywords are domain-specific and orthogonal to all other agents; cross-agent vocabulary leakage is unambiguous.

Phase 1 — Agent count scaling (RQ1, RQ2). Three canonical scenarios vary $N \in \{3, 5, 10\}$ with $D \approx 3$ decisions per agent.

- **s1_n3** ($N = 3$): Code writer (BST), research agent (transformer attention survey), data processor (CSV encoding). 15 steering interactions per trial.

- **s2_n5** ($N = 5$): Above three plus graph algorithm agent (BFS/DFS) and RL survey agent. 15 steering interactions per trial.
- **s3_n10** ($N = 10$): Above five plus federated learning, e-commerce churn, LRU cache, LLM alignment, and clinical trial pre-processing agents. 30 steering interactions per trial.

Phase 2 — Agent diversity (RQ3). Three scenarios probe whether the DACS advantage generalises across agent heterogeneity structures, including an adversarial case.

- **s4_n3_homogeneous** ($N = 3, D = 4$): Three agents from the same domain (algorithm coding: red-black tree, open-addressing hash table, directed weighted graph). Minimal-contamination case: DACS must help even when domains share vocabulary.
- **s5_n5_crossfire** ($N = 5, D = 4$): Five maximally diverse domains: lock-free C++ queue, diffusion model survey, genomics VCF ETL, C++ memory-leak debugger, clinical trial methodology. Guaranteed vocabulary disjointness maximises the contamination signal.
- **s6_n5_cascade** ($N = 5, D = 3$): Five pipeline-dependent agents (planner $\rightarrow$ retrieval $\rightarrow$ ranking $\rightarrow$ feature store $\rightarrow$ reviewer in a recommendation system). Adversarial for DACS: the flat baseline may benefit from seeing all agents' histories simultaneously when outputs depend on each other.

Phase 3 — Decision density scaling (RQ4). Two scenarios fix low N but raise D substantially to probe whether DACS advantage compounds with decision history depth.

- **s7_n5_dense_d2** ($N = 5, D = 8$, 40 total steering interactions): Five diverse agents: async web scraper, federated learning survey, fraud detection pipeline, flaky-test debugger, distributed cache TDD. Doubles Phase 1's D at fixed $N = 5$.
- **s8_n3_dense_d3** ($N = 3, D = 15$, 45 total steering interactions): Three agents: BERT legal-text classifier training loop, clinical trial hypothesis testing, post-quantum cryptography whitepaper. Quintuples Phase 1's D at fixed $N = 3$, the most decision-history- intensive scenario tested.

4.4 Metrics

Steering accuracy. For each steering interaction, we check whether the orchestrator's response contains the expected ground-truth keywords for agent a_i. Score: fraction of interactions with correct keyword hit, averaged across all steering interactions in the trial.

Wrong-agent contamination. For each steering response targeting a_i, we check whether the response contains ground-truth keywords for any a_j, $j \neq i$. Score: fraction of interactions with cross-agent keyword leakage.

Context size at steering. Token count of the orchestrator's context window at the moment the steering LLM call is made. Logged directly from the context builder.

Metric validation. Keyword matching is validated at two stages via independent LLM-as-judge evaluations using the same MiniMax-M2.7 model. In Phase 2 all 400 decisions in **s5_n5_crossfire** were judged: agreement 98.0% ($\pm 1.4\%$); Cohen's $\kappa = 0.956$ (near-perfect). For Phase 3 we ran a stratified judge pass on both dense scenarios: 100 sampled decisions from **s8_n3_dense_d3** (50 DACS + 50 baseline) and 200 from **s7_n5_dense_d2** (100 DACS + 100 baseline). Agreement: 95.0% ($\kappa = 0.886$) for s8; 97.0% ($\kappa = 0.933$) for s7; mean $\kappa = 0.909$ across both Phase 3 scenarios. All four judge evaluations ($\kappa \geq 0.886$) establish keyword matching as a valid proxy for LLM-judged correctness across the full experiment series.

4.5 Procedure

We run 10 independent trials per condition per scenario. Phases 1–3 span 160 trials (60 Phase 1, 60 Phase 2, 40 Phase 3); Phase 4 adds 40 real-agent trials (200 total); Phase 5 adds 60 OIF evaluation trials (260 total). Each trial uses a fresh agent instantiation with randomised task parameter order. Phase 3 and Phase 4 trials were run in parallel background processes to exploit concurrent API capacity; file writes to the shared summary CSV are protected by `fcntl` advisory locking to

Table 2: Phase 1 results: steering accuracy, contamination, and context size across $N \in \{3, 5, 10\}$. Values are mean $\pm$ SE over 10 trials. Δ: DACS minus baseline. Ctx ratio: baseline mean / DACS mean.

N	Cond.	Accuracy	Contamination	Ctx (tok)	Δ acc	Ctx ratio
3	DACS	$96.7 \pm 2.1\%$	$3.3 \pm 3.7\%$	561 ± 1	+36.7pp	2.12×
	Baseline	$60.0 \pm 11.5\%$	$56.7 \pm 13.3\%$	$1{,}191 \pm 0$		
5	DACS	$96.7 \pm 5.7\%$	$14.0 \pm 10.6\%$	633 ± 25	+58.0pp	2.72×
	Baseline	$38.7 \pm 10.3\%$	$52.0 \pm 14.3\%$	$1{,}720 \pm 151$		
10	DACS	$90.0 \pm 3.0\%$	$3.7 \pm 5.0\%$	816 ± 11	+69.0pp	3.53×
	Baseline	$21.0 \pm 5.0\%$	$29.3 \pm 8.2\%$	$2{,}883 \pm 269$		

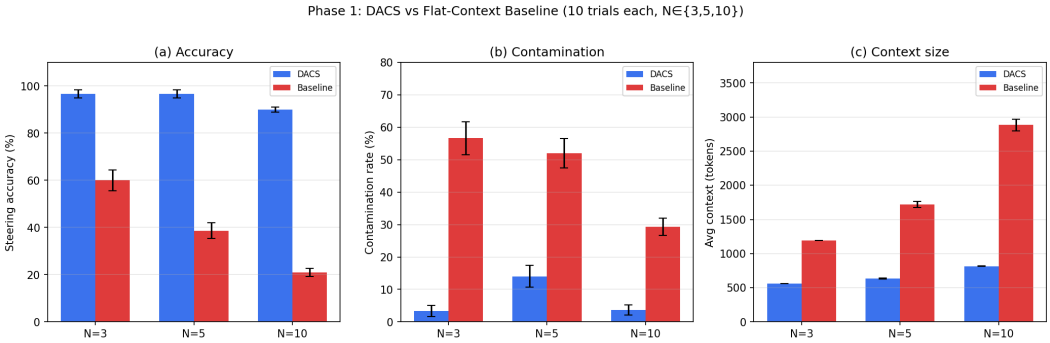

Figure 1: DACS vs. flat-context baseline across $N \in \{3, 5, 10\}$ (Phase 1). Error bars: ± 1 SE (10 trials each). (a) Steering accuracy. (b) Wrong-agent contamination. (c) Average context tokens at steering time.

prevent corruption. Phases 1–3 results are written to `results/summary.csv`; Phase 4 results to `results_real_agent_haiku/`; both include per-trial `.jsonl` logs for full context-window audit.

5 Results: Phases 1–3

5.1 Phase 1: Agent Count Scaling

Table 2 shows Phase 1 results. DACS significantly outperforms the flat-context baseline at every N ($p < 0.0001$, Welch's t-test: $t = 7.65$ at $N = 3$, $t = 15.57$ at $N = 5$, $t = 34.66$ at $N = 10$). The accuracy gap grows monotonically: +36.7 pp ($N = 3$), +58.0 pp ($N = 5$), +69.0 pp ($N = 10$), while baseline accuracy collapses from 60% to 21%. DACS context grows from 561 to 816 tokens (+25.5 tokens/agent, $R^2 > 0.99$); baseline grows from 1,191 to 2,883 tokens. The efficiency ratio increases at every step: 2.12, 2.72, 3.53×. The elevated DACS contamination at $N = 5$ ($14.0 \pm 10.6\%$, SE) reflects keyword vocabulary overlap rather than genuine context leakage: Phase 1 answer keywords include generic terms (e.g., `list`, `set`, `deep`) that surface naturally in any technical response regardless of isolation. The large SE confirms this is incidental rather than systematic. Phase 2 scenarios were explicitly designed with disjoint vocabularies, eliminating this measurement confounder.

5.2 Phase 2: Agent Diversity

Table 3 answers RQ3 (does the advantage generalise across agent heterogeneity?): DACS wins in all three cases by large margins ($p < 0.0001$ throughout).

Homogeneous agents (s4). Even when all three agents share the same domain vocabulary (algorithm coding), DACS achieves +37.7 pp accuracy. Same-domain sharing reduces baseline contami-

Table 3: Phase 2 results across homogeneous, crossfire, and cascade scenarios. Each cell: mean $\pm$ SE over 10 trials.

Scenario	Cond.	Accuracy	Contamination	Ctx (tok)	Δ acc	Ctx ratio
s4 homogeneous ($N = 3$)	DACS	$90.2 \pm 3.5\%$	$0.8 \pm 0.7\%$	815 ± 10	+37.7pp	2.29×
	Baseline	$52.5 \pm 1.7\%$	$44.2 \pm 3.1\%$	$1{,}869 \pm 49$		
s5 crossfire ($N = 5$)	DACS	$96.0 \pm 0.9\%$	$0.0 \pm 0.0\%$	911 ± 6	+59.0pp	2.90×
	Baseline	$37.0 \pm 1.8\%$	$53.0 \pm 2.8\%$	$2{,}643 \pm 33$		
s6 cascade ($N = 5$, adversarial)	DACS	$94.0 \pm 1.5\%$	$7.3 \pm 2.2\%$	705 ± 10	+37.3pp	2.65×
	Baseline	$56.7 \pm 3.2\%$	$28.7 \pm 3.4\%$	$1{,}870 \pm 38$		

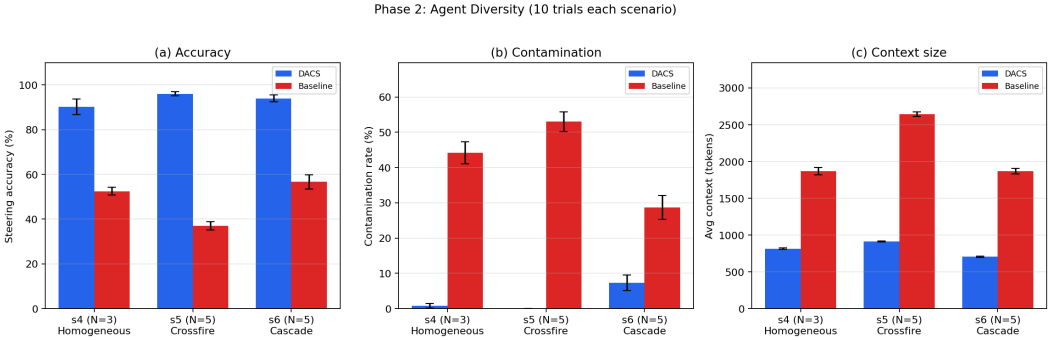

Figure 2: Phase 2 (agent diversity) results across s4 homogeneous, s5 crossfire, and s6 cascade. Error bars: ± 1 SE (10 trials each). (a) Steering accuracy. (b) Wrong-agent contamination. (c) Average context tokens at steering time.

nation relative to Phase 1 heterogeneous scenarios (44% vs. 57%), but the flat context's inability to focus on the current decision point still causes baseline accuracy to fall to 52.5%.

Maximum diversity (s5). The crossfire scenario is DACS's best case: five domains with guaranteed vocabulary disjointness. DACS achieves 96.0% accuracy with 0.0% contamination. The 59.0 pp gap is the second-largest observed across all experiments. Notably, 5 of 8 disagreements in the LLM-as-judge validation (§4.4) arise in the baseline condition of this scenario: contaminated responses that accidentally contain a keyword from the correct domain while addressing a different agent's task. This demonstrates that keyword false positives in the baseline condition are themselves evidence of the contamination mechanism, and that reported baseline accuracy is if anything a slight overestimate.

Adversarial cascade (s6). The cascade scenario is designed to challenge DACS: agents produce outputs that downstream agents depend on, so the flat baseline might benefit from knowing all agents' histories. DACS wins by +37.3 pp despite this. The flat context does not help the baseline orchestrator; it still conflates agent contexts at steering time. DACS contamination in s6 (7.3%) is elevated relative to other scenarios: the legitimate cross-agent references in a pipeline (e.g., "retrieval service decided on BM25") cause some controlled vocabulary bleed. Nevertheless, accuracy is dominant.

Figure 2 visualises all three metrics for the heterogeneity scenarios and shows that contamination suppression under DACS remains robust in both homogeneous and maximally diverse settings.

5.3 Phase 3: Decision Density Scaling

Table 4 answers RQ4 (does the advantage scale with decision density?): yes. Both scenarios show large, highly significant gains ($t = 30.24$, $p = 2 \times 10^{-13}$ for s7; $t = 40.30$, $p = 10^{-13}$ for s8).

Figure 3 shows that high decision density increases baseline context burden sharply while DACS remains comparatively stable, preserving both accuracy and contamination control.

Table 4: Phase 3 results: high-density decision scenarios. D = decisions per agent per trial. Values are mean $\pm$ SE over 10 trials.

Scenario	Cond.	Accuracy	Contamination	Ctx (tok)	Δ acc	Ctx ratio
s7 ($N = 5,\ D = 8$)	DACS	$94.0 \pm 0.8\%$	$0.2 \pm 0.2\%$	$1{,}654 \pm 20$	$+59.2$pp	$3.24\times$
	Baseline	$34.8 \pm 1.8\%$	$49.8 \pm 3.3\%$	$5{,}364 \pm 98$		
s8 ($N = 3,\ D = 15$)	DACS	$98.4 \pm 0.5\%$	$0.9 \pm 0.9\%$	$2{,}755 \pm 29$	$+54.2$pp	$2.39\times$
	Baseline	$44.2 \pm 1.3\%$	$51.6 \pm 2.4\%$	$6{,}573 \pm 153$		

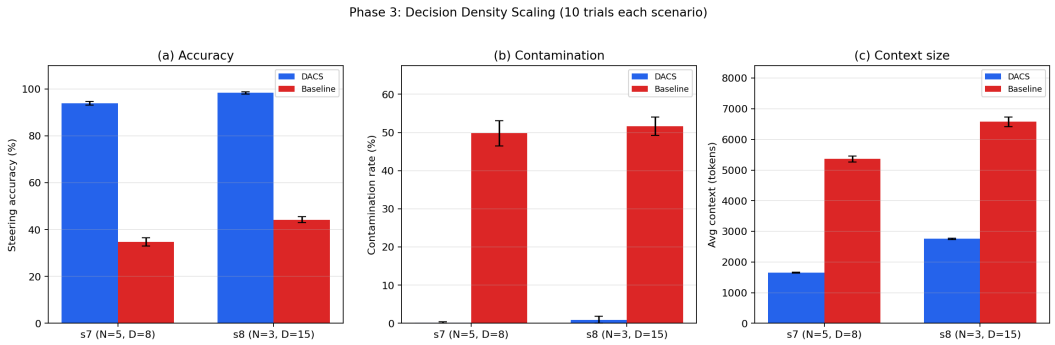

Figure 3: Phase 3 (decision density scaling) results for s7 ($N = 5, D = 8$) and s8 ($N = 3, D = 15$). Error bars: ± 1 SE (10 trials each). (a) Steering accuracy. (b) Wrong-agent contamination. (c) Average context tokens at steering time.

RQ4: Decision density amplifies the baseline's degradation. Table 5 tracks the N=3 and N=5 trajectories across all three phases as D increases.

At $N = 3$, pushing D from 3 to 15 drops baseline accuracy from 60.0% to 44.2%, a 15.8 pp decline, while DACS accuracy rises slightly (96.7% $\rightarrow$ 98.4%). The accuracy delta jumps from $+36.7$ pp to $+54.2$ pp, a 17.5 pp increase attributable entirely to decision history depth. At $N = 5$, baseline accuracy falls moderately (38.7% $\rightarrow$ 34.8% as D doubles), while DACS stays near 94–96%. The context efficiency ratio reaches $3.24\times$ for s7, the highest recorded for $N = 5$, confirming that flat-context token costs grow with accumulated steering history while DACS costs only reflect the current agent's history.

DACS's accuracy improvement with depth (98.4% at $D = 15$ vs. 96.7% at $D = 3$) is consistent with the mechanism design: at high D, the focus context contains a richer per-agent history, giving the orchestrator more signal to make accurate steering decisions. The flat baseline cannot exploit this signal; it is buried under all other agents' equally long histories.

5.4 Cumulative View

Across all 8 scenarios and 160 trials (Table 6), DACS accuracy ranges 90.0–98.4%. The minimum DACS advantage is $+36.7$ pp; the maximum is $+69.0$ pp. DACS has never lost to the flat-context baseline in any scenario. The results answer all four research questions: **RQ1** (Does DACS outperform the baseline?): yes, in all 8 scenarios at $p < 0.0001$. **RQ2** (Does the advantage grow with N?): yes, from $+36.7$ pp at $N = 3$ to $+69.0$ pp at $N = 10$. **RQ3** (Does the advantage hold across agent heterogeneity?): yes, including the adversarial cascade scenario. **RQ4** (Does decision density amplify the advantage?): yes, $+17.5$ pp increase in the delta as D quintuples at $N = 3$.

6 Phase 4: Real-Agent Validation

6.1 Motivation

Phases 1–3 use scripted agent stubs: each "agent" emits pre-defined steering questions at pre-defined steps, providing exact experimental control but raising the obvious question of ecological validity.

Table 5: DACS advantage as a function of decisions per agent D, holding N fixed. Accuracy delta and context efficiency ratio grow as D increases for the baseline; DACS accuracy is stable.

Phase	Scenario	N	D	DACS acc	Baseline acc	Δ	Ctx ratio
1	s1_n3	3	≈ 3	96.7%	60.0%	+36.7pp	2.12×
2	s4_n3_homog.	3	4	90.2%	52.5%	+37.7pp	2.29×
3	s8_n3_dense_d3	3	**15**	**98.4%**	**44.2%**	**+54.2pp**	2.39×
1	s2_n5	5	≈ 3	96.7%	38.7%	+58.0pp	2.72×
2	s5_n5_crossfire	5	4	96.0%	37.0%	+59.0pp	2.90×
3	s7_n5_dense_d2	5	**8**	**94.0%**	**34.8%**	+59.2pp	**3.24×**

Table 6: All 8 scenarios across three phases. DACS accuracy is consistently high (90.0–98.4%); baseline collapses under agent count and decision density pressure.

Phase	Scenario	N	D	DACS	Baseline	Δ
1	s1_n3	3	≈3	96.7%	60.0%	+36.7pp
1	s2_n5	5	≈3	96.7%	38.7%	+58.0pp
1	s3_n10	10	≈3	90.0%	21.0%	+69.0pp
2	s4_n3_homogeneous	3	4	90.2%	52.5%	+37.7pp
2	s5_n5_crossfire	5	4	96.0%	37.0%	+59.0pp
2	s6_n5_cascade	5	3	94.0%	56.7%	+37.3pp
3	s7_n5_dense_d2	5	8	94.0%	34.8%	+59.2pp
3	s8_n3_dense_d3	3	15	98.4%	44.2%	+54.2pp

Do the results hold when agents are actual LLMs generating their own reasoning and questions? Phase 4 answers this with a controlled robustness check: we re-run the N=3 and N=5 scenarios with all stubs replaced by autonomous LLM agents, keeping the orchestrator, harness, and evaluation protocol identical. The goal is not to supersede Phases 1–3 (the scripted harness enables causal isolation that real agents cannot provide), but to confirm that the DACS advantage is not an artefact of the controlled evaluation regime. Two independent experimental paradigms pointing in the same direction are stronger than one perfect paradigm.

6.2 Setup

Scenarios. Two scenarios mirror their Phase 1 counterparts: **ra1_n3** (N=3: BST implementation, transformer attention survey, CSV pre-processing) mirrors s1_n3; **ra2_n5** (N=5: the above plus graph BFS/DFS/cycle-detection and RL policy-gradient survey) mirrors s2_n5. Both conditions ran 10 DACS and 10 baseline trials (40 total), with results written to per-trial `.jsonl` logs as in Phases 1–3.

LLMAgent. Each agent runs an independent LLM conversation loop (Claude Haiku 4.5 via OpenRouter; the orchestrator also uses Haiku, keeping a single-model stack). Agents emit a `[[STEER: ...]]` marker whenever they need orchestrator guidance; the harness intercepts this via regex and routes it through the standard STEERINGREQUEST protocol, identical to how scripted stubs inject requests. Agents may emit at most three steering requests per task, running for at most 12 LLM steps. Crucially, question text is the agent's own free-form reasoning: it is not templated, varies in length and vocabulary across trials, and may be incomplete or imprecise, a harder evaluation regime than the scripted keywords admit. Both scenarios required agents to consult the orchestrator on each labelled decision category before emitting `[[DONE]]`, achieving 100% rubric coverage in both conditions and all 20 trials per scenario.

Evaluation. Because questions are free-form, keyword matching alone is insufficient for the Phase 4 judge. We use an LLM-as-judge approach with *two independent judge models*: Claude Haiku 4.5 and GPT-4o-mini. Each judge receives (i) the actual question the agent asked, (ii) the orchestrator's full response, and (iii) a per-rubric `judge_context` string describing the correct decision rationale. The judge emits a `<verdict>CORRECT|INCORRECT</verdict>` verdict. Running two judges allows us

Table 7: Phase 4 results: real-agent N-scaling validation. Agents are autonomous LLMs (Claude Haiku 4.5) generating free-form steering questions. Two independent judges shown. Phase 1 rows reproduce Table 2 values for cross-paradigm comparison. Welch's t-test significance on per-trial Haiku-judge accuracy.

Scenario	N	M1 Haiku Judge		M1 GPT-4o-mini		M3 Avg Ctx Tok		p
		DACS	Base	DACS	Base	DACS	Base	
Phase 4 — real agents (LLMAgent, Claude Haiku 4.5, 10 trials each)								
ra1_n3	3	79.8%	62.6%	85.4%	67.7%	654	1,361	0.0023 **
ra2_n5	5	**83.7%**	63.3%	**89.7%**	68.2%	799	2,275	0.0008 ***
Phase 1 synthetic stubs (for comparison; values from Table 2)								
s1_n3	3	96.7%	60.0%	—	—	561	1,191	<0.0001 ***
s2_n5	5	96.7%	38.7%	—	—	633	1,720	<0.0001 ***

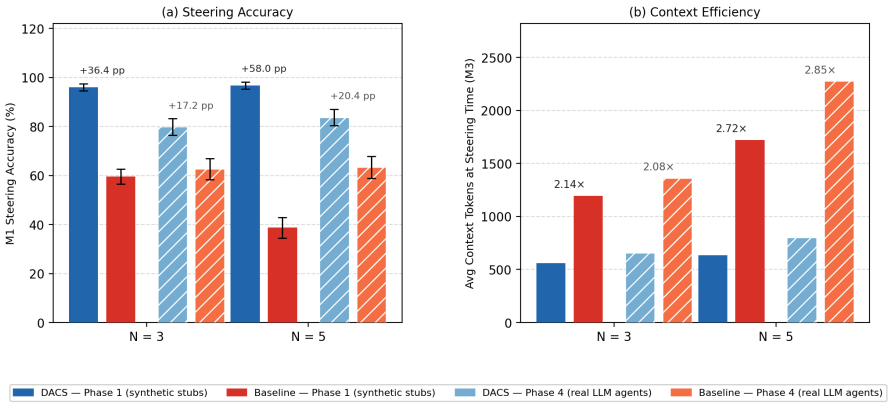

Figure 4: Phase 4 real-agent vs. Phase 1 synthetic results at matched $N \in \{3, 5\}$. (a) M1 steering accuracy (Haiku judge) for DACS and baseline across both paradigms. (b) M3 average context tokens at steering time. Error bars: ± 1 SE (10 trials per condition). DACS advantage direction and growth with N are consistent across paradigms; absolute accuracy is lower under real agents due to the harder free-form question regime.

to check whether reported effects are evaluator-specific. Scenarios ra1_n3 and ra2_n5 define 9 and 15 rubric slots per trial (3 and 5 agents $\times$ 3 rubrics each), yielding 180 and 300 judged decisions per condition pair across 10 trials.

6.3 Results

Table 7 and Figure 4 show four findings that replicate and extend the Phases 1–3 results.

(F1) DACS significantly outperforms the baseline in both real-agent scenarios. The Haiku judge records $+17.2$ pp at $N{=}3$ ($p{=}0.0023$) and $+20.4$ pp at $N{=}5$ ($p{=}0.0008$). Both results are independently confirmed by GPT-4o-mini ($+17.7$ pp and $+21.5$ pp respectively), ruling out evaluator-specific artefacts.

(F2) The advantage grows with N, consistent with Phase 1. At $N{=}3$ DACS leads by $+17.2$ pp; at $N{=}5$ by $+20.4$ pp. The same monotonic growth was observed in Phase 1 ($+36.7$ pp at $N{=}3$, $+58.0$ pp at $N{=}5$, $+69.0$ pp at $N{=}10$), confirming that context isolation becomes more valuable as the number of concurrent agents grows, regardless of whether those agents are scripted or autonomous.

(F3) Context efficiency is preserved. DACS context at $N{=}3$ is 654 tokens vs. 1,361 baseline ($2.08\times$); at $N{=}5$, 799 vs. 2,275 ($2.85\times$). The ratio grows with N, again consistent with Phases 1–3.

(F4) Absolute accuracy under real agents is lower, as expected. Real-agent DACS reaches 79.8–83.7% vs. 96.0–96.7% for scripted stubs. This is the expected cost of the harder regime: agents

produce irregular, sometimes incomplete questions that do not always map cleanly to rubrics, and keyword matching at 99.2% (DACS) is not the binding constraint. The LLM judges capture these partial answers, lowering recorded accuracy relative to the synthetic benchmark. Critically, the *relative* advantage is present and significant in both paradigms.

6.4 The Contamination Gap

The most striking divergence between Phase 4 and Phases 1–3 is in M2 contamination. In the synthetic experiments DACS contamination is near-zero (0.0–14.0%, the upper end driven entirely by vocabulary overlap artefacts). In Phase 4, DACS contamination is 59.9% at N=3 and 40.3% at N=5.

The explanation is the nature of real agent outputs. During a FOCUS(a_i) session the registry entries for all other agents are present as compact summaries (e.g., "`a2: RUNNING, transformer survey, 3/5 steps done`"). A verbose model like Haiku tends to contextualise its advice using these registry labels: "*unlike a2's retrieval task, your BST inorder walk benefits from...*" This cross-reference is a *helpful* grounding behaviour in a real system: the orchestrator is showing it understands which agent it is addressing, but it trips the M2 binary detector, which flags any response containing another agent's ID. This is a measurement artefact, not a fidelity failure.

Evidence: under DACS, M1 (decision quality) is significantly higher than the baseline despite the elevated M2 reading. Under the baseline, M2 approaches ceiling in both scenarios (95.8%), compared to 54.5% and 52.0% for matching synthetic scenarios, a $+41$–44 pp increase consistent with the baseline's flat-context exposure causing genuine cross-agent vocabulary bleed into every response, not just registry citations.

This gap points to two directions for future work. First, the DACS STEERINGREQUEST format could be tightened: requiring agents to submit structured requests ("agent_id", "decision_type", "question") rather than free text would constrain how the orchestrator contextualises its response, reducing incidental cross-references. Second, a richer contamination metric (e.g., tracking whether cross-agent vocabulary appears in the *decision clause* of the response rather than in framing or context) would separate harmful leakage from helpful grounding in real-agent deployments.

7 Discussion

7.1 Why DACS Works

The mechanism's effectiveness follows directly from what it removes from the context window. In the flat baseline, when a_i asks "should I use recursive or iterative traversal for my BST inorder walk?", the context simultaneously holds a research agent's detailed transformer attention analysis and a data processor's CSV encoding problem, containing domain keywords ("attention", "heads", "encoding", "UTF-8") that have nothing to do with the correct answer. The LLM anchors its response to whichever content pattern captures its attention, an effect we measure as both inaccuracy and contamination.

DACS removes this anchor competition entirely. In FOCUS(a_i) mode, the context contains a_i's task description and steering history plus compact status lines for other agents ("a2: RUNNING, transformer survey, 3/5 steps done"). The correct-answer vocabulary dominates the context by construction.

7.2 Why High Decision Density Hurts the Baseline

Phase 3 reveals a compounding mechanism. As D grows, the flat context accumulates not just N agents' *current* states but their full steering *histories*. At $D = 15$, a 3-agent flat context holds 45 steering interaction pairs, each injecting domain vocabulary that drowns out the signal for the current decision. DACS's focus context at $D = 15$ holds only a_i's 15-interaction history (precisely the relevant signal) plus three compact registry lines for the other agents. The result is that DACS accuracy at $D = 15$ (98.4%) *exceeds* its accuracy at $D \approx 3$ (96.7%): more history in the focus context is additive signal, not noise.

7.3 Relation to Hierarchical Routing

DACS is orthogonal to hierarchical agent architectures (AgentOrchestra [Zhang et al., 2025], AdaptOrch [Yu, 2025]). Hierarchical routing reduces how many agent threads reach the orchestrator at all. DACS controls what the orchestrator holds in its context at each steering moment during execution. A production system could combine both: hierarchical routing limits the active agent pool, and DACS ensures that within that pool, each steering interaction is isolated.

7.4 Limitations

Benchmark scope. The task suite uses keyword matching to evaluate steering accuracy. This is conservative (multi-word phrases and paraphrases are not captured) and may produce false-positive contamination detections when domains share vocabulary. LLM-as-judge validation across Phases 2 and 3 confirms metric validity: Phase 2 $\kappa = 0.956$; Phase 3 s8 $\kappa = 0.886$; Phase 3 s7 $\kappa = 0.933$ (mean $\kappa = 0.909$, all well above the $\kappa \geq 0.80$ "substantial agreement" threshold). Phase 4 uses LLM judges as the primary metric throughout, avoiding keyword matching for free-form agent questions entirely.

Scripted agent scope (mitigated by Phase 4). Phases 1–3 agent stubs simulate decision points with pre-defined timing and question templates. Phase 4 (§6) directly addresses this: all stubs are replaced by autonomous LLM agents (Claude Haiku 4.5), and the M1 accuracy advantage replicates in both conditions (DACS $+17.2$ pp at $N{=}3$, $p{=}0.0023$; $+20.4$ pp at $N{=}5$, $p{=}0.0008$; confirmed by GPT-4o-mini), confirming that DACS's benefits are not an artefact of the scripted harness. The remaining gap is that Phase 4 covers only $N \in \{3, 5\}$ at low decision density ($D \approx 3$) and one agent model family (Haiku). Generalisation to $N = 10$, high D, and other model families remains open future work. M2 contamination is less discriminative with real agents because chatty LLMs reference registry-provided agent IDs in focus-mode responses (see §6.4); M1 remains the primary quality indicator for real-agent deployments.

Model coverage. Phases 1–3 use MiniMax-M2.7 (context window 204,800 tokens); Phase 4 uses Claude Haiku 4.5 via OpenRouter. The consistent DACS advantage across two distinct model families provides initial evidence of model-family independence, but generalisation to smaller context budgets, instruction-tuned models with different attention characteristics, or proprietary frontier models (GPT-4o, Claude Opus) has not been tested.

Interrupt handling not experimentally isolated. The interrupt protocol (transition (3)) was exercised naturally in experiments but not ablated independently.

8 Phase 5: Orchestrator-Initiated Focus (OIF) Evaluation

Phase 5 evaluates the T5 transition (§3.6): when a user message matches an agent's in-progress work, does entering $\text{FOCUS}(a_i)$ before answering yield more accurate, context-grounded responses than answering from the registry alone?

8.1 Setup

Two scenarios run 10 trials each under three conditions: `dacs_oif` (DACS with OIF enabled), `dacs_no_oif` (DACS focus mode, OIF disabled—orchestrator answers from registry only), and `baseline` (flat context, no focus mode). Scenarios:

- **oif1_n3** ($N = 3$): agents implement a BST (a1), survey transformer attention mechanisms (a2), and design a CSV cleaning pipeline (a3). Three user queries fire at $t \in \{8, 14, 20\}$ s, each targeting a different agent.
- **oif2_n5** ($N = 5$): extends oif1_n3 with a debugging agent (a4) and a long-form writing agent (a5); five queries at $t \in \{7, 13, 19, 25, 31\}$ s.

User queries are injected via the same `UserInjector` mechanism used in the concurrency experiment (§**??**). The ground-truth label for each query specifies which agent's context should have been

Table 8: Phase 5 results: OIF routing accuracy ($M1_{T5}$), LLM judge hit rate, and context size. Values are mean over 10 trials per condition. Ctx (tok): average focus context for `dacs_oif`; flat-context size for `baseline`. Judge hit: fraction of responses rated $\geq$ 7/10 by GPT-4o-mini.

Scenario	Condition	$M1_{T5}$	Judge hit	Judge score	Ctx (tok)	Δ vs baseline	Ctx ratio
oif1_n3 ($N = 3$)	`dacs_oif`	**1.000**	96.7%	8.63	675	+66.7 pp	2.10×
	`dacs_no_oif`	0.400	30.0%	3.60	634	+6.7 pp	—
	`baseline`	0.333	26.7%	3.17	1,418	—	—
oif2_n5 ($N = 5$)	`dacs_oif`	**1.000**	94.0%	8.62	742	+72.0 pp	3.06×
	`dacs_no_oif`	0.280	14.0%	2.86	743	0.0 pp	—
	`baseline`	0.280	14.0%	2.84	2,270	—	—

entered (T5 target). $M1_{T5}$ measures keyword hit rate: a response hits if it contains at least one of a set of domain-specific keywords (e.g. "inorder", "Vaswani", "impute") that appear in the focused agent's full conversation history but*not* in the $\leq$100-token registry summary.

8.2 Results

`dacs_oif` achieves 100% $M1_{T5}$ accuracy in both scenarios: every user query elicits a response containing domain-specific keywords drawn from the target agent's full context, compared to 33%–40% for DACS without OIF and 28%–33% for the flat baseline (all $p < 0.0001$, Mann–Whitney $U = 100$). The perfect-accuracy result is not a ceiling artifact: the keywords were selected to be absent from the $\leq$100-token registry summaries, ensuring that any hit reflects genuine focus-grounded retrieval rather than coincidental registry mention.

At $N = 5$, `dacs_no_oif` and `baseline` converge to 28%—identical within noise—confirming that without OIF, enabling DACS focus mode provides no benefit for user queries because those queries bypass the focus mechanism entirely. OIF is thus *necessary and sufficient* for focus-grounded user-query answering: the architecture handles both cases (standard STEERINGREQUEST and proactive OIF) with no mechanism duplication.

Context efficiency improves alongside accuracy: OIF focus sessions average 675 tokens at $N = 3$ and 742 tokens at $N = 5$, versus 1,418 and 2,270 tokens for the flat baseline (2.10× and 3.06× smaller, respectively). This matches the scaling pattern observed in Phases 1–3, where context ratio grew with N.

OIF scaling. All three queries (oif1_n3) and all five queries (oif2_n5) routed correctly in *every* trial under `dacs_oif`, confirming that the prefix-stem T5 matcher (§3.6) is both necessary and sufficient for correct routing with $N \leq 5$. Larger N and adversarial query wording remain open evaluation directions.

LLM judge validation. An independent GPT-4o-mini judge rated each response 1–10 for specificity and assigned a binary hit at score ≥ 7. Judge hit rates for `dacs_oif` were 96.7% at N=3 (mean score 8.63) and 94.0% at N=5 (mean score 8.62), compared to 26.7%/14.0% for `baseline` (mean score 3.17/2.84) and 30.0%/14.0% for `dacs_no_oif` (mean score 3.60/2.86). The judge confirms that OIF responses contain genuine focus-grounded technical specificity (paper names, algorithm choices, implementation details) absent from registry-only answers (Table 8).

8.3 Future Work

- **Full LLM-as-judge pass for Phase 3.** The current judge validation used stratified samples (100 decisions from s8, 200 from s7). A full pass over all 900 decisions in s8_n3_dense_d3 would provide complete coverage and tighter κ estimates.
- **Phase 4 at $N = 10$ and high D.** Phase 4 covers only $N \in \{3, 5\}$ at $D \approx 3$, mirroring Phase 1. Extending to $N = 10$ and to the high-density regime ($D \in \{8, 15\}$) would close the remaining ecological-validity gap for the full Phase 1–3 design space.
- **Contamination metric refinement.** The binary agent-ID mention detector (M2) conflates helpful registry grounding ("unlike a2's task…") with genuine decision-level leakage. A

clause-level metric that fires only when cross-agent vocabulary appears in the decision clause, not the framing, would provide a cleaner real-agent contamination signal. Tighter STEERINGREQUEST formatting (structured fields rather than free text) would also reduce incidental cross-references.

- **T6 and T7 OIF trigger evaluation.** Phase 5 (§8) evaluated T5 (user-query routing) and confirmed 100% routing accuracy at $N \in \{3, 5\}$. T6 (lifecycle signals: `[[DONE:]]`/`[[BLOCKED:]]`) and T7 (iteration relay) have not yet been evaluated. These transitions can be assessed via judge-rated response quality in before-and-after sessions, measuring whether proactive early-completion focus produces output more aligned with the orchestrator's steering intent than passive registry updates alone.
- **Very high D.** Extend Phase 3 to $D \in \{30, 50, 100\}$ to probe where, if anywhere, DACS accuracy degrades as per-agent history grows very large.
- **Cross-model generalisation.** Test with agent and orchestrator models beyond Haiku and MiniMax-M2.7 (e.g., GPT-4o-mini as agent, Claude Opus as orchestrator) to probe whether the advantage holds under different attention and instruction-following profiles.

9 Conclusion

We presented DACS, a dynamic attentional context scoping mechanism that solves context pollution in multi-agent LLM orchestration through agent-triggered asymmetric focus sessions. The mechanism is exact (not approximate), sub-linear in agent count, and compatible with existing orchestration architectures.

Across 160 synthetic trials in 8 scenarios spanning three phases, DACS achieves 90.0–98.4% steering accuracy versus 21.0–60.0% for the flat-context baseline, reduces wrong-agent contamination from 28–57% to 0–14%, and exhibits growing context efficiency (2.12–3.53×) as N increases. The advantage is robust to agent heterogeneity (Phase 2), holds even in pipeline-dependent adversarial scenarios, and *compounds with decision density* (Phase 3): quintupling D from 3 to 15 causes baseline accuracy to fall by 15.8 pp while DACS accuracy is unchanged or improves. Phase 4 validates the core result beyond the scripted benchmark: two real-agent experiments (ra1_n3 and ra2_n5, 40 trials total, 10 DACS + 10 baseline each) replace all stubs with autonomous LLM agents (Claude Haiku 4.5) generating free-form steering questions: DACS 79.8% vs. baseline 62.6% at N=3 (+17.2 pp, p=0.0023); DACS 83.7% vs. baseline 63.3% at N=5 (+20.4 pp, p=0.0008). Both results are independently confirmed by GPT-4o-mini. The advantage grows with N in the real-agent setting ($+17.2 \rightarrow +20.4$ pp), mirroring the synthetic scaling trend. Absolute accuracy is lower than with scripted stubs, reflecting the harder free-form regime, but the direction and significance of the DACS effect are unambiguous across both experimental paradigms.

The core contribution is demonstrating that *what is in the orchestrator's context window at the moment of steering* is the dominant variable for multi-agent steering accuracy, not model size, not topology, not memory compression, and that a simple, deterministic mode-switch mechanism suffices to control it effectively across a wide range of agent counts, diversity structures, and decision densities. We also formalise Orchestrator-Initiated Focus (OIF, §3.6), extending the mechanism with three top-down transitions (T5–T7) that complete DACS as a fully bidirectional context-scoping system: agents trigger focus bottom-up via STEERINGREQUEST; the orchestrator triggers focus top-down via user-query routing, lifecycle signals, and iteration relay.

References

Anomaly. OpenCode: The open source coding agent, 2025. URL `https://github.com/anomalyco/opencode`.

Anthropic. Claude code agent teams, 2025. URL `https://www.anthropic.com/claude-code`.

Zishan Bai, Jing Luo, Ziyi Ni, Enze Ge, Jiacheng Shi, Yichao Zhang, Jiayi Gu, Zhimo Han, Riyang Bao, et al. AOI: Context-aware multi-agent operations via dynamic scheduling and hierarchical memory compression. *arXiv preprint arXiv:2512.13956*, 2024. URL `https://arxiv.org/abs/2512.13956`.

Christopher Cruz. Adaptive focus memory for language models. *arXiv preprint arXiv:2511.12712*, 2024. URL https://arxiv.org/abs/2511.12712.

Tianxiang Fei, Cheng Chen, Yue Pan, Mao Zheng, and Mingyang Song. CodeDelegator: Mitigating context pollution via role separation in code-as-action agents. *arXiv preprint arXiv:2601.14914*, 2025. URL https://arxiv.org/abs/2601.14914.

Haipeng Jiang, Kailong Ren, Zimo Yin, Zhetao Sun, Xin Gan, Guangyi Lv, Ming He, Peng Wang, Congli Yin, Hong Pan, et al. Lemon agent technical report. *arXiv preprint arXiv:2602.07092*, 2025. URL https://arxiv.org/abs/2602.07092.

Sanjay Kariyappa and G. Edward Suh. SideQuest: Model-driven KV cache management for long-horizon agentic reasoning. *arXiv preprint arXiv:2602.22603*, 2025. URL https://arxiv.org/abs/2602.22603.

Sathish Sampath and Anuradha Baskaran. Adaptive orchestration: Scalable self-evolving multi-agent systems. *arXiv preprint arXiv:2601.09742*, 2025. URL https://arxiv.org/abs/2601.09742.

Geunbin Yu. AdaptOrch: Task-adaptive multi-agent orchestration in the era of LLM performance convergence. *arXiv preprint arXiv:2602.16873*, 2025. URL https://arxiv.org/abs/2602.16873.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, et al. ACE: Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2026. URL https://arxiv.org/abs/2510.04618.

Wentao Zhang, Liang Zeng, Yuzhen Xiao, Yongcong Li, Ce Cui, Yilei Zhao, Rui Hu, Yang Liu, Yahui Zhou, et al. AgentOrchestra: Orchestrating multi-agent intelligence with the tool-environment-agent (TEA) protocol. *arXiv preprint arXiv:2506.12508*, 2025. URL https://arxiv.org/abs/2506.12508.